IPSSI - Ecole d'Informatique, Web et Design

Formation : Concepteur Développeur d'Applications (CDA) - Bachelor 3 FullStack

NoExcuses

Plateforme de pilotage du mode de vie sportif et nutritionnel

Dossier de Projet

Cahier des charges, conception, développement et déploiement

Dillon AZAG

Bachelor 3 FullStack

Depot : github.com/dillon816/Noexcuses

Sommaire

Partie I - Présentation générale 4

1. Contexte métier 4
2. Objectifs du projet 4
3. Parties prenantes 4
4. Glossaire 5

Partie II - Cahier des charges fonctionnel 6

5. Périmètre fonctionnel 6
6. Exigences fonctionnelles 7
7. Exigences non-fonctionnelles 7
8. Critères de succès 7

Partie III - Contraintes et analyse des risques 9

9. Contraintes du projet 9
10. Analyse des risques 9

Partie IV - Méthodologie et organisation 10

11. Méthode de gestion de projet (Agile Scrum) 10
12. Planning prévisionnel 10
13. Outils de suivi et pilotage (Kanban) 11
14. Gestion du code source (Git) 11
15. CI/CD et démarche DevOps 11

Partie V - Conception UX / UI 12

16. Charte graphique 12
17. Responsive Design et considérations UX 12
18. Zoning et arborescence 13
19. Wireframes et maquettes haute fidélité 14

Partie VI - Modélisation des données 17

20. Méthode MERISE 17
21. Règles de gestion 17
22. Modèle Conceptuel de Données (MCD) 18
23. Modèle Logique de Données (MLD) 19
24. Modèle Physique de Données (MPD) 20
25. Justifications et données calculées 20
26. Recovery Score : algorithme 20
27. Qualité du modèle 20

Partie VII - Conception applicative (UML) 21

28. Vue d'ensemble de la conception UML 21

29. Diagramme de cas d'utilisation . . . 22
30. Diagramme de séquence : Connexion / Authentification JWT . . . 23
31. Diagramme de séquence : Ajouter un repas . . . 24
32. Diagramme de classes . . . 25
33. Diagramme technique (Controllers / Services / Repositories) . . . 26

Partie VIII - Architecture logicielle . . . 27

34. Exigences techniques . . . 27
35. Pattern MVC avec Symfony . . . 27
36. Architecture n-tiers . . . 27
37. Composants externes et API . . . 28

Partie IX - Réalisation et développement . . . 29

38. Organisation du code . . . 29
39. Principaux endpoints de l'API . . . 29
40. Modules développés . . . 30
41. Extrait de code représentatif . . . 32

Partie X - Sécurité . . . 34

42. Analyse des menaces OWASP . . . 34
43. Authentification : JWT, bcrypt et connexion déléguée . . . 34
44. CORS, validation et protection des données (RGPD) . . . 34
45. Principaux risques OWASP retenus . . . 35

Partie XI - Tests et qualité . . . 36

46. Stratégie de tests . . . 36
47. Tests unitaires (PHPUnit) . . . 36
48. Tests d'intégration . . . 36
49. Intégration continue . . . 37

Partie XII - Déploiement et mise en production . . . 38

50. Conteneurisation Docker . . . 38
51. Environnements de développement et production . . . 38
52. Déploiement et stratégie de mise en production . . . 38
53. Retour d'expérience DevOps . . . 39

Conclusion . . . 40

Annexes . . . 41

PARTIE I

Présentation générale

1. Contexte métier

NoExcuses est une application de suivi fitness et nutrition destinée à accompagner l'utilisateur dans sa progression personnelle et le maintien d'une discipline quotidienne. Le projet naît d'un constat simple : maintenir sur la durée un équilibre entre effort physique et nutrition demande de suivre de nombreux paramètres (repas, entraînements, sommeil, poids). La multiplication des outils d'information et la difficulté à mesurer concrètement ses résultats rendent ce suivi laborieux, et la motivation finit souvent par s'essouffler.

L'application répond à cette problématique en centralisant, dans une interface unique, le suivi nutritionnel et sportif de l'utilisateur. Elle transforme des données brutes (repas, poids, séances, sommeil) en indicateurs clairs et exploitables, afin de rendre visible la progression et de soutenir la régularité au quotidien. Le design privilégie la simplicité, la motivation et la lisibilité rapide des données.

2. Objectifs du projet

- **Centraliser le suivi** : regrouper en un seul point d'entrée l'ensemble des données liées à l'hygiène de vie (alimentation et activité physique).
- **Aider à la régularité** : simplifier la saisie quotidienne pour encourager l'utilisateur à maintenir ses objectifs sur le long terme.
- **Analyser la progression** : fournir une interface de visualisation de l'évolution des performances et de la forme physique.
- **Optimiser le temps** : automatiser les calculs nutritionnels et la gestion des programmes sportifs pour réduire la charge mentale de l'utilisateur.

3. Parties prenantes

Le projet, réalisé en solo dans le cadre du diplôme CDA, implique néanmoins plusieurs parties prenantes dont les rôles sont clairement identifiés :

Partie prenante	Rôle dans le projet	Attentes principales
Porteur du projet (MOA)	Définit le besoin et la vision produit	Un outil fidèle au besoin métier
Développeur (MOE) - Dillon AZAG	Conçoit, développe et déploie l'application	Solution robuste, maintenable, livrée dans les délais
Utilisateurs finaux	Sportifs amateurs suivant leur nutrition et leurs entraînements	Simplicité, rapidité de saisie, indicateurs clairs
Jury / formateurs IPSSI	Évaluent le projet aux jalons du diplôme	Respect du cahier des charges et des bonnes pratiques
Prestataires externes	Fournissent des services tiers (CalorieNinjas, Mistral AI, Google)	Disponibilité et fiabilité des API

4. Glossaire

Terme	Définition
MERISE	Méthode d'analyse et de conception de systèmes d'information (MCD, MLD, MPD).
MCD / MLD / MPD	Modèles Conceptuel / Logique / Physique de données : niveaux successifs de modélisation de la base.
MVP	Minimum Viable Product : produit minimum viable regroupant les fonctionnalités essentielles.
JWT	JSON Web Token : jeton signé prouvant l'identité de l'utilisateur sur une API sans session serveur.
bcrypt	Algorithme de hachage des mots de passe, lent et salé, résistant aux attaques par force brute.
OAuth 2.0	Protocole d'autorisation permettant de se connecter via un fournisseur tiers (ici Google).
OWASP	Organisation de référence recensant les principales vulnérabilités web (Top 10).
RGPD	Règlement général sur la protection des données personnelles.
API REST / SPA	Interface serveur sans état / application web monopage (Single Page Application, ici React).
MVC	Modèle-Vue-Contrôleur : patron d'architecture séparant données, présentation et logique.
ORM	Object-Relational Mapping : ici Doctrine, qui fait le lien entre objets PHP et tables SQL.
Docker / CI/CD	Conteneurisation de l'application / intégration et déploiement continus automatisés.
Tonnage	Charge totale soulevée sur une séance (répétitions × charge), indicateur de volume d'entraînement.
Recovery Score	Score de récupération (0-100) calculé à partir du sommeil, des calories et de la charge d'entraînement.
Macros	Macronutriments : protéines, glucides et lipides.

PARTIE II

Cahier des charges fonctionnel

5. Périmètre fonctionnel

L'application est organisée en six modules fonctionnels couvrant l'ensemble du besoin métier. Ces six modules constituent le périmètre du produit minimum viable (MVP).

5.1 Gestion des utilisateurs (Auth & Profil)

- Système d'authentification sécurisé : inscription et connexion par email / mot de passe (JWT), ou via un compte Google (OAuth 2.0).
- Gestion du profil utilisateur : informations personnelles (prénom, nom, sexe, taille, date de naissance).
- Paramétrage des objectifs personnels : calories cibles, macronutriments, poids visé et niveau d'activité.

5.2 Nutrition

- Moteur de recherche d'aliments s'appuyant sur une API nutritionnelle externe (CalorieNinjas), avec traduction FR/EN assurée par l'IA Mistral.
- Journal alimentaire quotidien : ajout, modification et suppression de repas.
- Calcul automatique des apports (calories et macronutriments) à partir des quantités saisies.

5.3 Entraînement

- Modèles de séances réutilisables (Push, Pull, Legs…) que l'utilisateur prépare une fois puis démarre en un clic.
- Déroulé clair : une séance démarrée est « en cours » et reste modifiable, puis passe « terminée » une fois validée.
- Séances composées d'exercices et de séries (répétitions, charge), avec calcul automatique du tonnage (volume soulevé) par série et par séance.
- Historique des séances réellement effectuées : seules les séances terminées alimentent les statistiques et le Recovery Score.

5.4 Recovery Budget

- Outil d'aide à la décision calculant un score de récupération (0-100) à partir des données de sommeil, du bilan calorique et de la charge des séances terminées sur les 7 derniers jours.
- Recommandation adaptée : repos, séance légère ou séance normale.

5.5 Dashboard

- Synthèse quotidienne : calories consommées par rapport à l'objectif, macros, série de jours consécutifs (streak).
- Alerte de récupération lorsque le score est faible.

5.6 Progression

- Visualisation de l'évolution du poids dans le temps.

- Statistiques de suivi nutritionnel et de charge d'entraînement (volume hebdomadaire et tendance) pour analyser les tendances.

6. Exigences fonctionnelles

Les exigences fonctionnelles décrivent ce que le système doit permettre de faire.

Réf.	Exigence fonctionnelle	Priorité
EF01	L'utilisateur peut créer un compte et se connecter (email/mot de passe ou Google).	Haute
EF02	L'utilisateur peut renseigner et modifier son profil et ses objectifs.	Haute
EF03	L'utilisateur peut rechercher un aliment et l'ajouter à son journal du jour.	Haute
EF04	Le système calcule automatiquement les calories et macros consommées.	Haute
EF05	L'utilisateur peut créer des modèles de séances réutilisables, les démarrer, puis suivre exercices et séries.	Haute
EF06	Le système calcule le tonnage d'une séance.	Moyenne
EF07	Le système calcule un score de récupération et une recommandation.	Moyenne
EF08	L'utilisateur visualise sa synthèse du jour sur un tableau de bord.	Haute
EF09	L'utilisateur consulte l'évolution de son poids et ses statistiques.	Moyenne
EF10	L'utilisateur peut exporter ses données personnelles (JSON, Excel) au titre du RGPD.	Moyenne

7. Exigences non-fonctionnelles

Les exigences non-fonctionnelles décrivent la manière dont le système doit se comporter (qualité de service).

Réf.	Catégorie	Exigence
ENF01	Sécurité	Mots de passe hachés (bcrypt), routes protégées par JWT, conformité OWASP.
ENF02	Performance	Temps de réponse moyen inférieur à 1 seconde pour les requêtes courantes.
ENF03	Ergonomie	Interface responsive (Mobile First), utilisable sur mobile et ordinateur.
ENF04	Maintenabilité	Code structuré en couches (MVC), testé et documenté.
ENF05	Portabilité	Application conteneurisée (Docker), déployable de façon reproductible.
ENF06	Conformité	Respect du RGPD sur les données personnelles et de santé.

8. Critères de succès

Le projet NoExcuses sera considéré comme une réussite lorsque l'ensemble des critères suivants sera satisfait :

- **Complétude fonctionnelle** : l'ensemble des fonctionnalités du MVP est implémenté, opérationnel et conforme au cahier des charges, sans fonctionnalités bloquantes ou incomplètes.

- **Validation technique et tests** : les fonctionnalités principales font l'objet de tests unitaires et d'intégration, et l'ensemble des tests automatisés est validé sans erreur dans la chaîne d'intégration continue.
- **Performance du système** (objectif visé) : temps de réponse moyen inférieur à une seconde pour les requêtes courantes (consultation du dashboard, ajout d'un repas ou d'une séance).
- **Portabilité et conteneurisation** : l'application est déployable et exécutable dans un environnement Dockerisé, garantissant la reproductibilité.
- **Expérience utilisateur** : l'interface est fonctionnelle, ergonomique et responsive, conçue pour une prise en main intuitive.
- **Sécurité et conformité** : les données sont correctement sécurisées (hachage, contrôle des accès, protection OWASP).
- **Maintenabilité et qualité documentaire** : le code est structuré et documenté, et la documentation technique et fonctionnelle est complète et claire.

PARTIE III

Contraintes et analyse des risques

9. Contraintes du projet

9.1 Contraintes temporelles

Le projet s'inscrit dans un calendrier strict de six mois, nécessitant des livraisons de jalons mensuels validant chaque étape du cycle de développement (conception, développement, tests, déploiement). Chaque jalon conditionne le passage au suivant.

9.2 Contraintes techniques

La stack technique est imposée par le référentiel de la formation : back-end Symfony (API PHP), front-end React (SPA), base de données SQL (MySQL), conteneurisation Docker et chaîne d'intégration continue GitHub Actions. L'architecture doit respecter une séparation claire des responsabilités et les bonnes pratiques de sécurité (hachage des mots de passe, protection OWASP, conformité RGPD).

10. Analyse des risques

Les principaux risques identifiés sur le projet, avec leur probabilité, leur impact et les mesures d'atténuation mises en place :

Risque	Prob.	Impact	Mesure d'atténuation
Indisponibilité ou limitation d'une API externe (CalorieNinjas, Mistral)	Moyenne	Moyen	Mise en cache locale des aliments déjà recherchés ; repli sur un dictionnaire de traduction si l'IA est indisponible.
Responsive Design : interface inutilisable sur mobile	Moyenne	Moyen	Conception Mobile First, navigation adaptée (barre inférieure sur mobile), tests sur plusieurs tailles d'écran.
Fuite de données sensibles (identifiants, santé)	Faible	Élevé	Hachage bcrypt, JWT, secrets hors dépôt Git, conformité OWASP et RGPD.
Dérive du planning (projet solo)	Moyenne	Élevé	Découpage Agile en jalons mensuels, suivi Kanban, priorisation du MVP.
Différences entre environnements (dev/prod)	Moyenne	Moyen	Conteneurisation Docker garantissant un environnement identique et reproductible.

PARTIE IV

Méthodologie et organisation

11. Méthode de gestion de projet (Agile Scrum)

Le projet est conduit selon une méthodologie Agile Scrum adaptée à un projet individuel. Le développement est organisé en sprints mensuels correspondant aux jalons pédagogiques du cursus ; chaque sprint comprend une phase de planification, de développement, de validation et d'amélioration continue.

Dans ce contexte solo, le développeur cumule les rôles de Product Owner et de Scrum Master : il pilote le backlog, définit les priorités et valide les incréments. Les rituels Scrum sont adaptés : un journal de bord hebdomadaire remplace les réunions quotidiennes, et chaque jalon mensuel joue le rôle de Sprint Review et de Sprint Retrospective. Cette approche permet une livraison progressive des fonctionnalités tout en conservant une forte capacité d'adaptation.

12. Planning prévisionnel

Le planning du projet est formalisé sous forme de diagramme de Gantt, réparti sur les six jalons du diplôme (analyse et cahier des charges, méthodologie et UX, modélisation de la base, conception et architecture, développement, tests et sécurité, déploiement et soutenance).

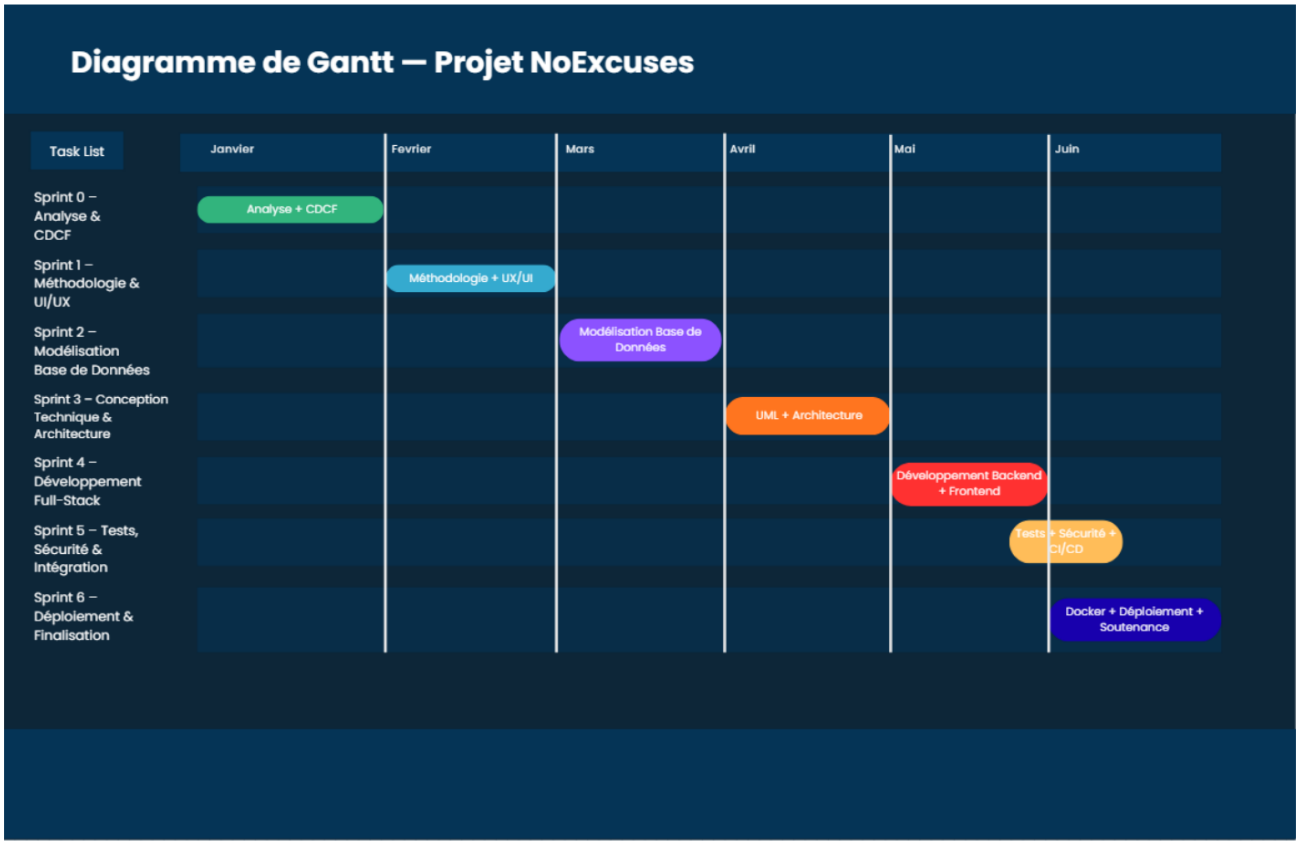


Fig. 1 - Diagramme de Gantt du projet NoExcuses

13. Outils de suivi et pilotage (Kanban)

Le suivi du projet est réalisé à l'aide de GitHub Projects, permettant la mise en place d'un tableau Kanban directement lié au dépôt de code source. Les tâches sont organisées sous forme d'Issues et réparties dans cinq colonnes :

- **Backlog** : liste globale des fonctionnalités et tâches à réaliser ;
- **À faire** : tâches planifiées pour le sprint en cours ;
- **En cours** : tâches actuellement en développement ;
- **En validation** : tâches terminées en attente de vérification ;
- **Terminé** : fonctionnalités validées et intégrées au projet.

Cette organisation assure une traçabilité complète entre les fonctionnalités développées, les commits Git et l'avancement global du projet. Une revue hebdomadaire permet d'évaluer l'état d'avancement, de mettre à jour le tableau et d'identifier d'éventuels retards ou difficultés techniques.

14. Gestion du code source (Git)

La gestion du code source est assurée avec Git, hébergé sur GitHub. Git permet le suivi des évolutions du code, la gestion des versions, la sécurisation du développement grâce à l'historique des modifications et une organisation professionnelle inspirée des pratiques en entreprise.

14.1 Stratégie de branches

Une stratégie de type Git Flow simplifiée est adoptée, reposant sur deux branches principales :

- **main** : contient les versions stables correspondant aux livrables validés lors des jalons pédagogiques ;
- **develop** : branche principale d'intégration continue des nouvelles fonctionnalités, sur laquelle se fait le développement courant.

Même dans un contexte de développement individuel, les intégrations vers **main** passent par des Pull Requests afin de simuler une revue de code et de garantir la qualité. Des versions du projet sont créées à chaque étape majeure via un système de tags Git (v0.9 bêta, v1.0 finale), permettant d'identifier clairement les jalons du développement.

15. CI/CD et démarche DevOps

Dans une logique DevOps, une chaîne d'intégration continue (GitHub Actions) automatise les contrôles de qualité du code et sécurise les intégrations. Elle se déclenche à chaque push sur la branche develop et à chaque Pull Request vers main, et exécute :

- **Back-end (Symfony / PHP)** : installation des dépendances (composer install), génération des clés JWT, migrations, exécution des tests unitaires et d'intégration via PHPUnit.
- **Front-end (React / Node)** : installation des dépendances, puis build de production (npm run build) pour valider la compilation.

Une version qui échoue aux tests ne peut pas être déployée, ce qui garantit la stabilité de l'application à chaque jalon. Cette chaîne constitue la première brique d'un déploiement continu, complété par la conteneurisation Docker (voir Partie XII).

PARTIE V

Conception UX / UI

16. Charte graphique

16.1 Concept visuel

L'identité visuelle repose sur trois principes fondamentaux : **simplicité d'utilisation**, **motivation permanente** et **lisibilité rapide des données**. Le design adopte une approche moderne et minimaliste, inspirée des applications professionnelles de suivi sportif et des interfaces SaaS orientées performance, afin de mettre en avant les informations essentielles sans surcharge visuelle.

16.2 Palette de couleurs

La palette repose sur des couleurs sobres assurant une lecture claire des indicateurs, avec un vert énergie comme couleur d'accentuation symbolisant la progression et la réussite.

Usage	Couleur	Code
Vert énergie (accent, réussite, actions)	Vert	#22C55E
Noir principal (navigation latérale)	Noir	#111111
Bleu-noir secondaire (contrastes)	Ardoise	#0F172A
Fond d'interface	Gris clair	#F1F5F9
Surfaces et cartes	Blanc	#FFFFFF
Texte secondaire / atténué	Gris	#64748B / #94A3B8
Notification d'alerte	Orange	#EA580C

16.3 Typographie

La police retenue est **Inter**, choisie pour sa lisibilité élevée sur mobile et desktop et sa cohérence avec les standards actuels. Une hiérarchie typographique stricte a été définie : titres de page (Inter Bold 30 px), titres de section (Bold 18 px), sous-titres (Bold 14 px), texte courant (Regular 16 px), labels (Medium 14 px) et indicateurs numériques (Medium/Bold 22 px).

16.4 Style d'interface et icônes

L'interface adopte un design épuré privilégiant la compréhension immédiate : cartes d'information pour structurer les contenus, mise en avant des indicateurs clés, réduction du texte au profit d'éléments visuels (indicateurs circulaires de calories, barres de progression des macros, boutons d'actions rapides). Les icônes suivent un style monochrome minimaliste, avec une représentation claire par module (Dashboard, Nutrition, Entraînements, Recovery, Progression, Profil).

17. Responsive Design et considérations UX

L'application est conçue selon une approche **Mobile First**. Sur desktop, l'interface repose sur une navigation latérale associée à une zone de contenu centrale ; sur mobile, une navigation inférieure inspirée des applications natives permet un accès rapide aux fonctionnalités principales.

L'expérience utilisateur a été pensée pour une utilisation intuitive et rapide au quotidien : organisation claire des informations pour limiter la charge cognitive, mise en avant des indicateurs essentiels, retours visuels explicites (couleurs d'état, indicateurs de progression) et respect des standards ergonomiques des applications mobiles et dashboards modernes.

18. Zoning et arborescence

Le zoning définit la structure des écrans : en-tête, menu latéral de navigation permanent (desktop) ou navigation basse (mobile) et zone de contenu principal.

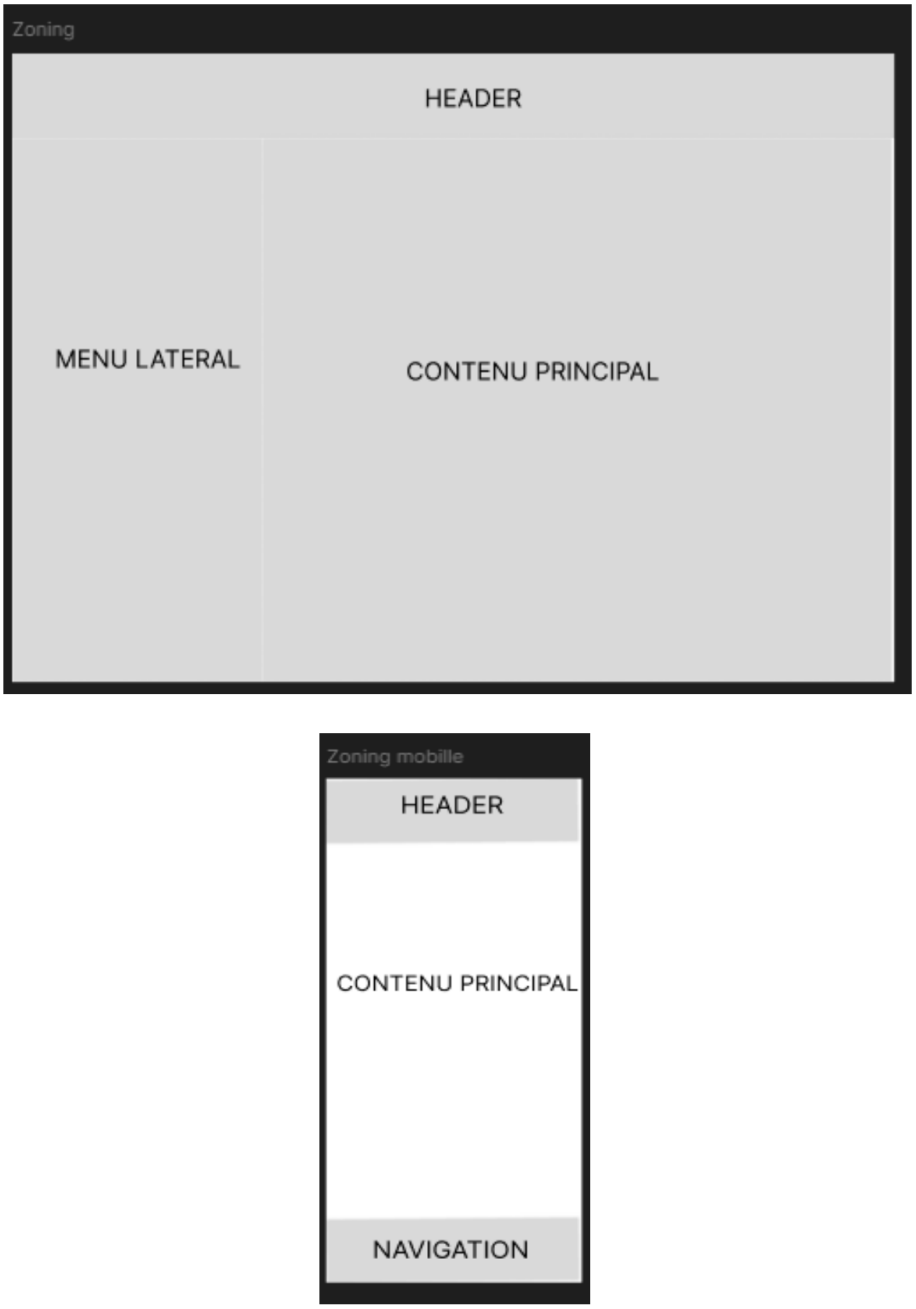


Fig. 2 - Zoning des écrans (desktop et mobile)

Le plan du site (sitemap) organise les écrans autour du tableau de bord, point d'entrée après connexion, qui donne accès aux modules Nutrition, Entraînement, Recovery, Progression et Profil.

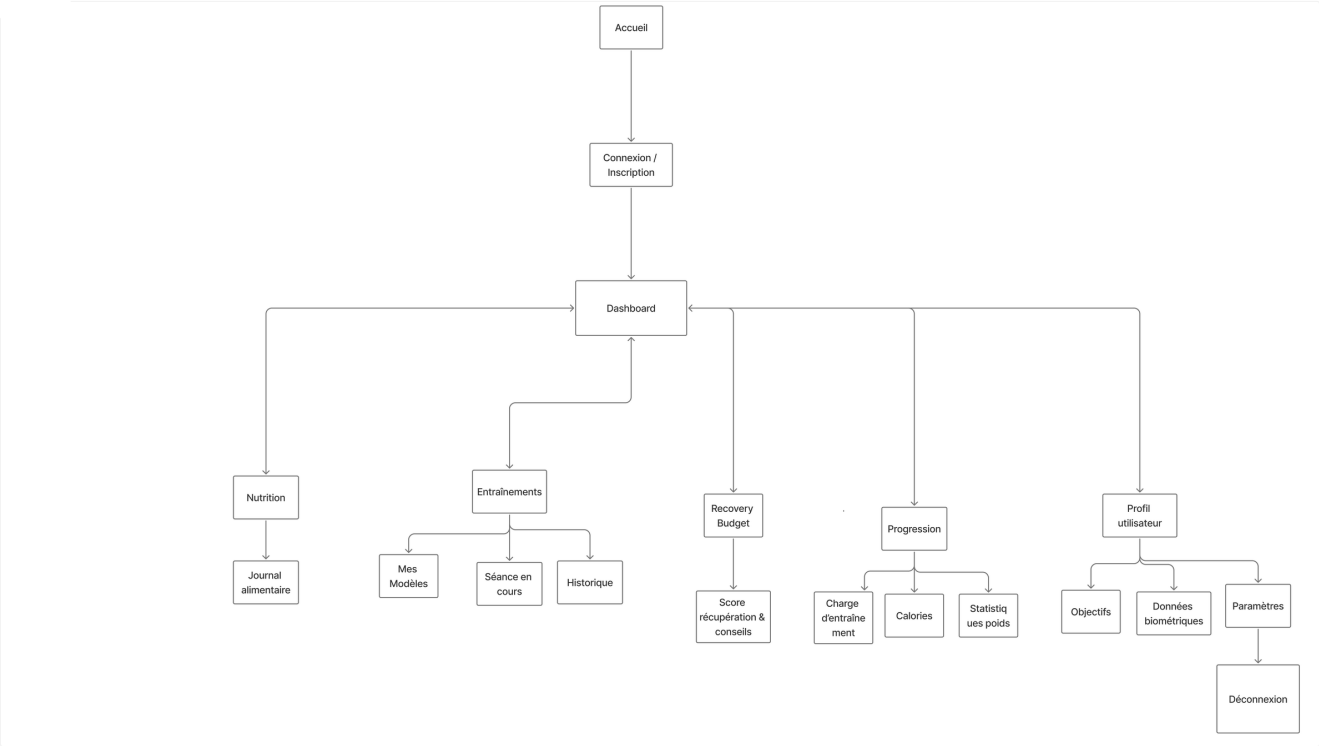


Fig. 3 - Plan du site (sitemap)

19. Wireframes et maquettes haute fidélité

Les wireframes basse fidélité posent la structure des écrans avant la mise en forme graphique.

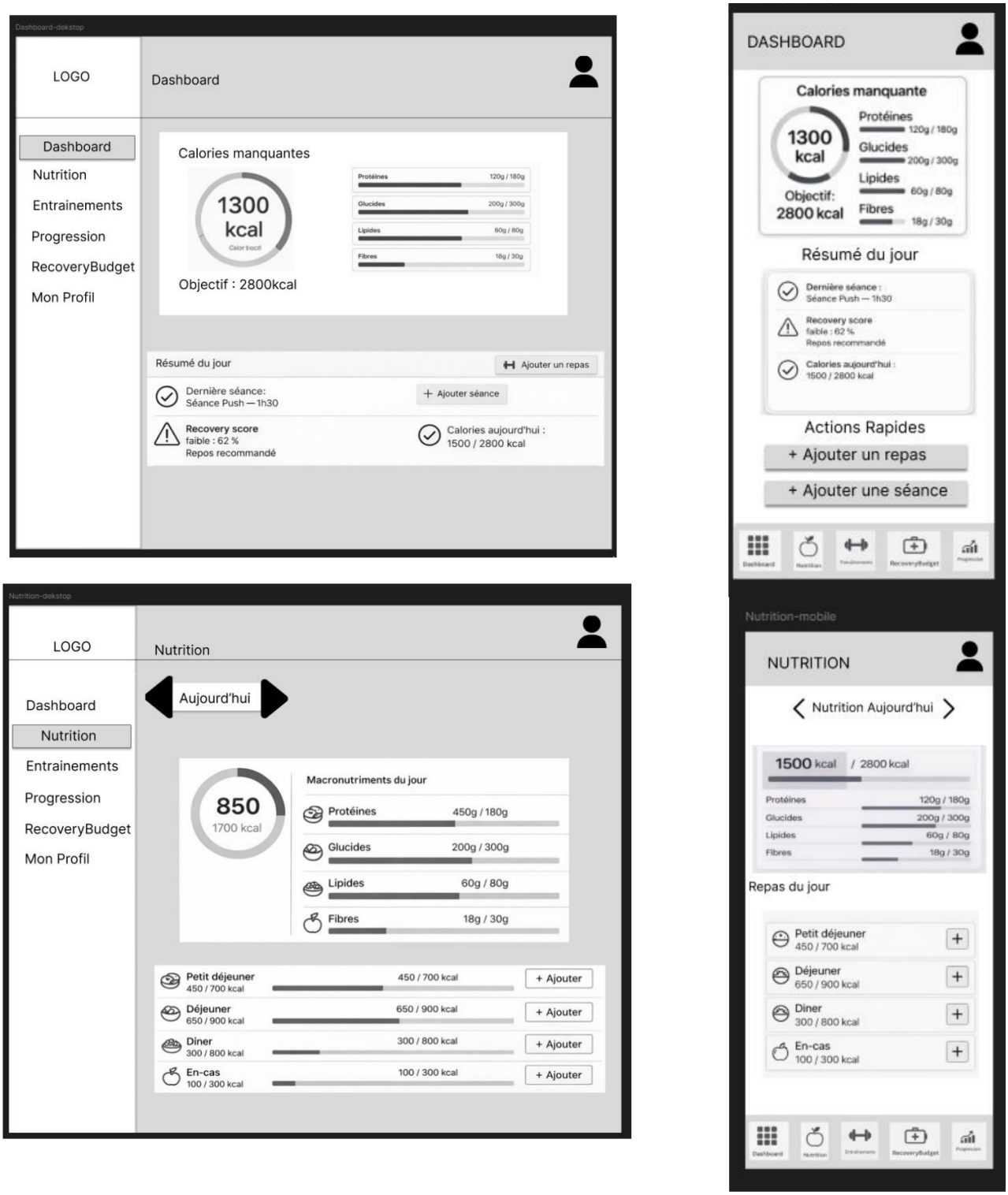


Fig. 4 - Wireframes basse fidélité

Les maquettes haute fidélité concrétisent la charte graphique sur les écrans clés, en versions desktop et mobile.

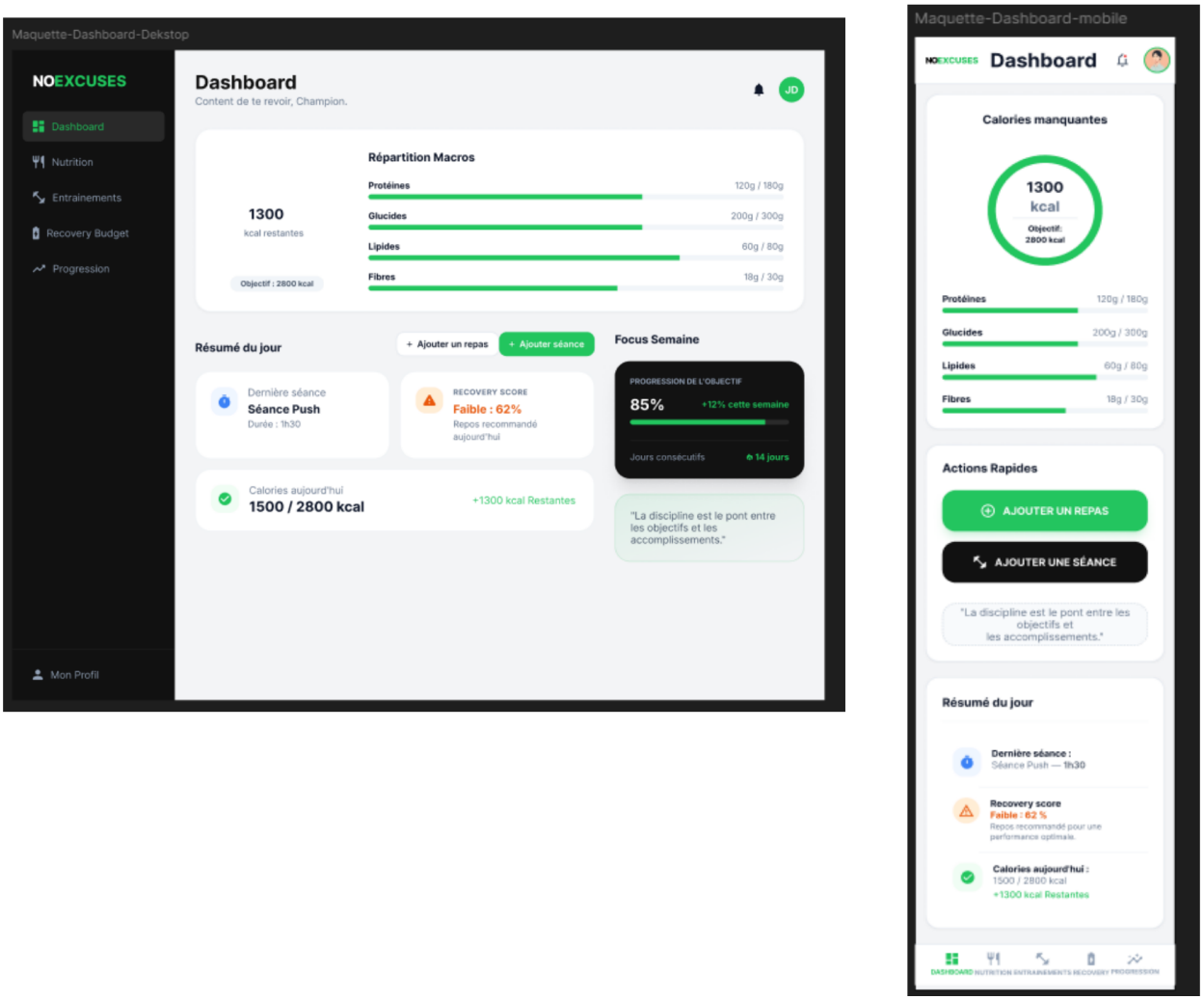


Fig. 5 - Maquettes haute fidélité (dashboard)

PARTIE VI

Modélisation des données

20. Méthode MERISE

La modélisation de la base de données a été réalisée avec la méthode MERISE, qui structure la conception en trois étapes successives : le Modèle Conceptuel de Données (MCD, représentant les entités métier et leurs relations), le Modèle Logique de Données (MLD, traduction en tables relationnelles) et le Modèle Physique de Données (MPD, implémentation concrète dans le SGBD). Cette progression garantit que la base répond aux besoins fonctionnels tout en restant cohérente et optimisée.

21. Règles de gestion

Les règles de gestion formalisent les contraintes métier que le modèle de données doit garantir :

Réf.	Règle de gestion
RG01	Un utilisateur est identifié de manière unique par son adresse email.
RG02	Un utilisateur peut définir plusieurs objectifs ; un objectif appartient à un seul utilisateur.
RG03	Un journal alimentaire est unique par utilisateur et par jour (une seule date par journal).
RG04	Un journal contient plusieurs lignes ; chaque ligne référence un aliment du référentiel.
RG05	Un aliment peut être référencé par plusieurs lignes de journal (référentiel partagé).
RG06	Une séance appartient à un seul utilisateur et se compose de plusieurs séries.
RG07	Une série référence un exercice du référentiel et appartient à une seule séance.
RG08	Un enregistrement de sommeil est unique par utilisateur et par nuit.
RG09	Un score de récupération est unique par utilisateur et par jour.
RG10	La suppression d'un utilisateur entraîne la suppression en cascade de toutes ses données.
RG11	Les totaux nutritionnels et le tonnage sont des données calculées, recalculées à chaque modification.

22. Modèle Conceptuel de Données (MCD)

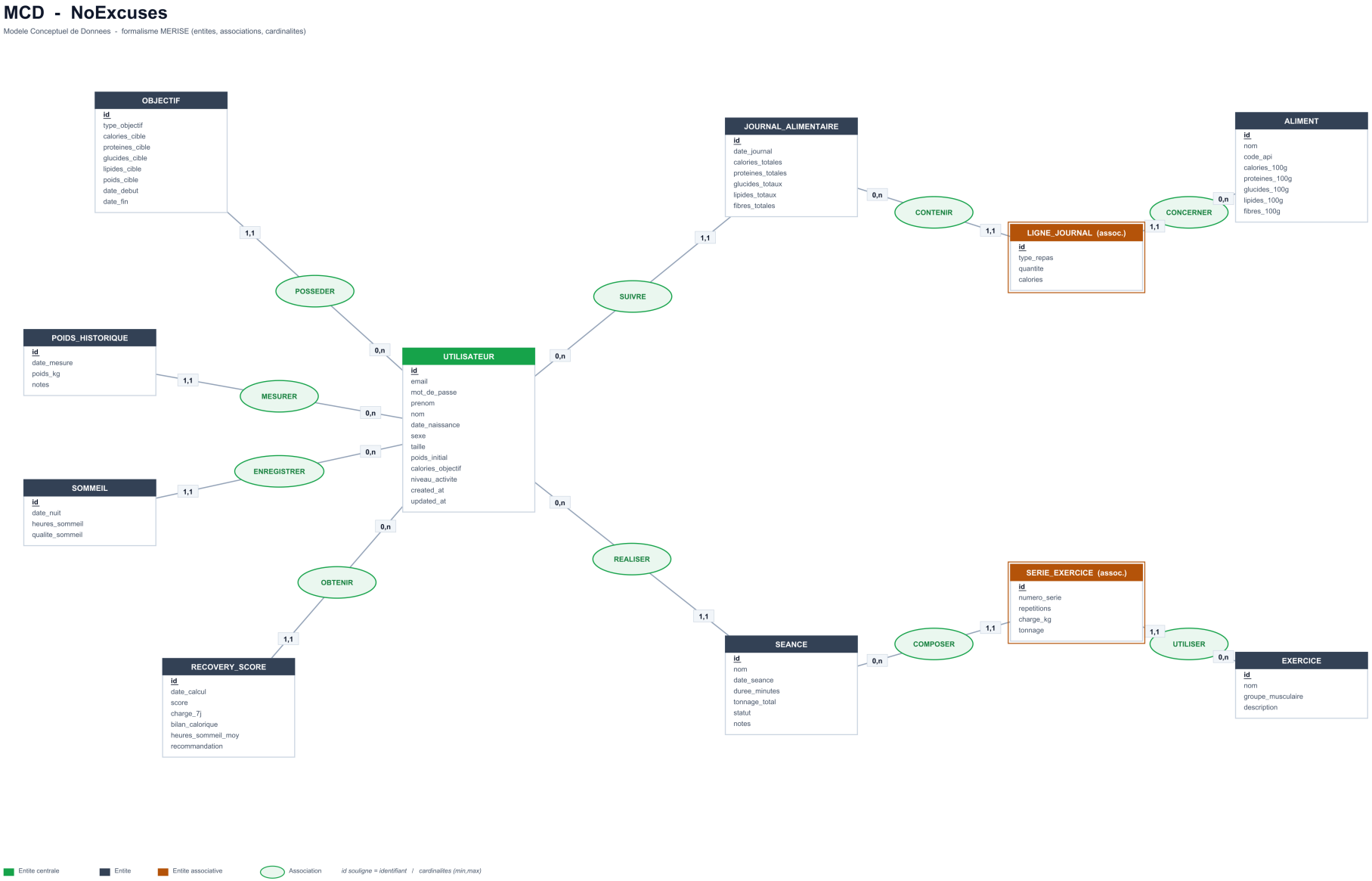


Fig. 6 - Modèle Conceptuel de Données (formalisme MERISE)

23. Modèle Logique de Données (MLD)

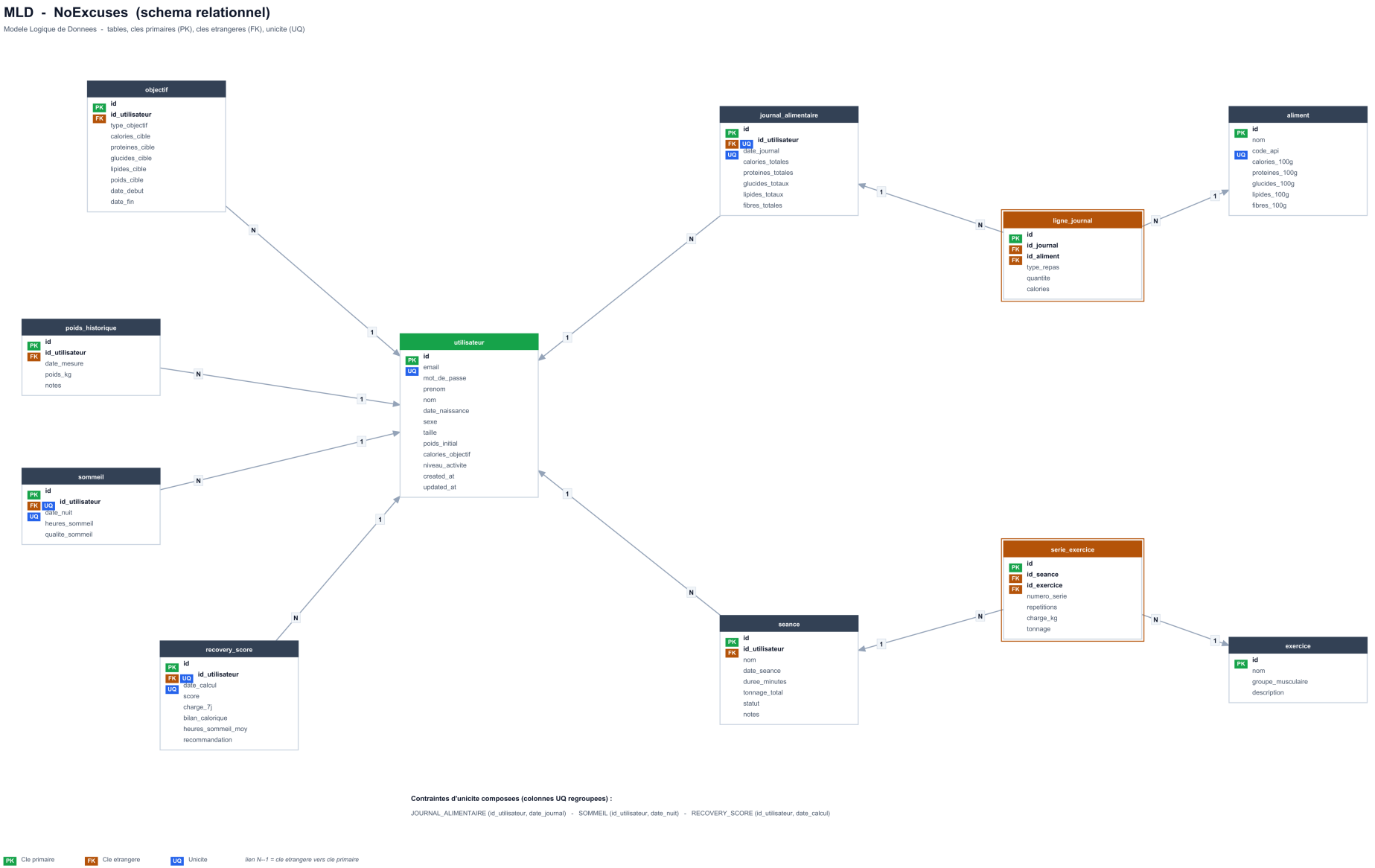


Fig. 7 - Modèle Logique de Données (schéma relationnel : PK / FK / UQ)

24. Modèle Physique de Données (MPD)

Le MPD correspond à l'implémentation concrète dans le SGBD MySQL. Les tables sont définies avec des types SQL adaptés (INT, VARCHAR, DECIMAL, DATE, DATETIME), des clés primaires auto-incrémentées, des clés étrangères avec contraintes d'intégrité, des contraintes d'unicité et des règles de suppression : **ON DELETE CASCADE** pour les données possédées par l'utilisateur, **ON DELETE RESTRICT** pour les référentiels partagés (Aliment, Exercice). Le dictionnaire de données complet et le script SQL de création figurent en annexe.

25. Justifications et données calculées

Le modèle a été conçu autour de l'entité centrale **Utilisateur**, qui regroupe l'ensemble des informations liées à l'hygiène de vie. Les différents modules (nutrition, entraînement, sommeil, récupération, progression) ont été séparés afin d'améliorer la lisibilité, d'éviter les redondances et de faciliter l'évolution future. Les relations plusieurs-à-plusieurs ont été résolues par des tables intermédiaires (ligne_journal entre journal et aliment, serie_exercice entre séance et exercice).

Certaines données sont **calculées** afin de faciliter l'affichage et les performances : calories_totales, protéines_totales, glucides_totaux, lipides_totaux, tonnage_total et recovery_score. Elles sont dérivées des valeurs nutritionnelles des aliments (pour 100 g), des quantités consommées et des données d'entraînement et de sommeil, puis mises à jour automatiquement par la logique applicative à chaque ajout ou modification.

26. Recovery Score : algorithme

Le score de récupération est un élément clé du projet. Il est calculé sur 100 points à partir de trois composantes, puis converti en recommandation (repos, séance légère ou séance normale) :

Composante	Poids	Principe
Sommeil	40 pts	Maximum autour de 8 h ; pénalité forte en dessous de 5 h.
Bilan calorique	30 pts	Maximum proche de l'objectif ; pénalité en cas de déficit important.
Charge des séances terminées (7 j)	30 pts	Maximum pour une charge modérée ; pénalité en cas de surcharge.

Ce score guide l'utilisateur dans le choix de l'intensité de ses séances et l'aide à optimiser sa progression tout en évitant le surentraînement.

27. Qualité du modèle

Le modèle respecte les principes de normalisation (troisième forme normale, 3NF), ce qui permet d'éviter les redondances, de garantir la cohérence des données et de faciliter la maintenance et l'évolution du système.

PARTIE VII

Conception applicative (UML)

28. Vue d'ensemble de la conception UML

La conception applicative s'appuie sur plusieurs diagrammes UML complémentaires, qui décrivent le comportement et la structure du système avant le développement. Le **diagramme de cas d'utilisation** présente les fonctionnalités offertes aux acteurs ; les **diagrammes de séquence** détaillent les scénarios critiques (authentification, ajout d'un repas) ; le **diagramme de classes** formalise la structure orientée objet ; et le **diagramme technique** montre l'organisation en couches de l'API.

Ces diagrammes assurent la cohérence entre le besoin fonctionnel (Partie II), le modèle de données (Partie VI) et l'implémentation. Chacun est présenté en grand format dans les pages suivantes, accompagné d'une description de son rôle.

29. Diagramme de cas d'utilisation

Diagramme de cas d'utilisation - NoExcuses



Fig. 8 - Diagramme de cas d'utilisation

30. Diagramme de séquence : Connexion / Authentification JWT

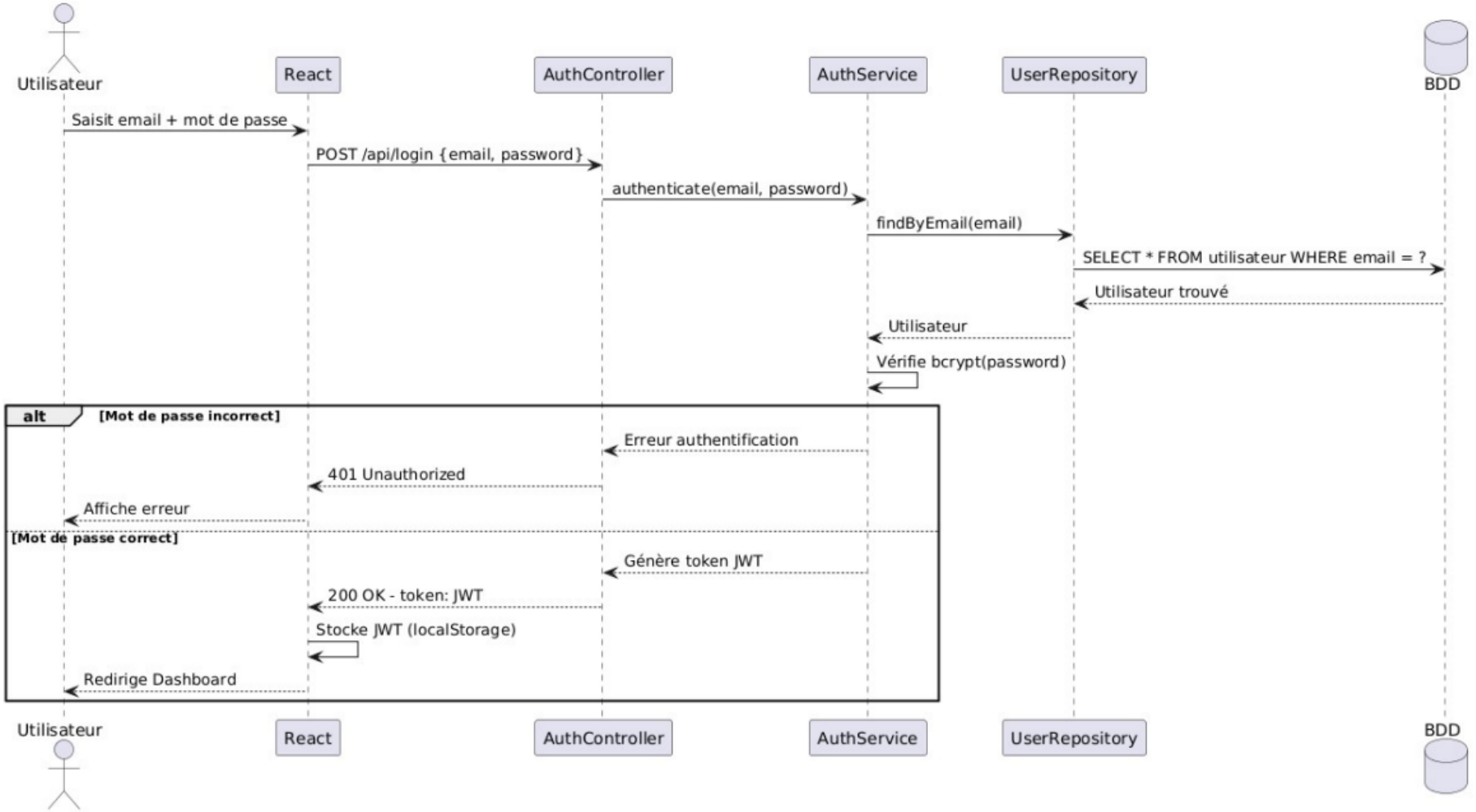


Fig. 9 - Séquence : Connexion / Authentification JWT

31. Diagramme de séquence : Ajouter un repas



Fig. 10 - Séquence : Ajouter un repas

32. Diagramme de classes

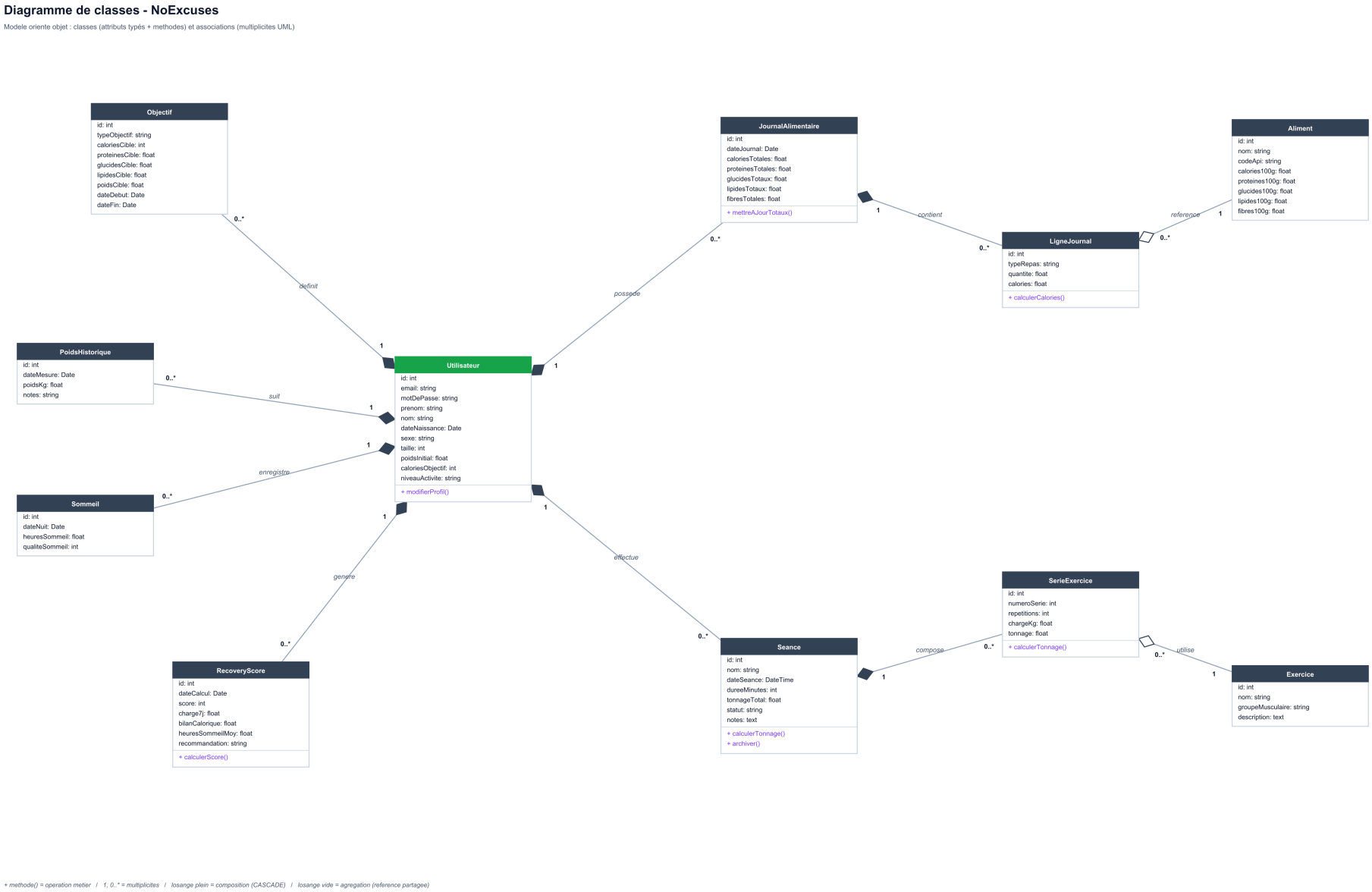


Fig. 11 - Diagramme de classes

33. Diagramme technique (Controllers / Services / Repositories)

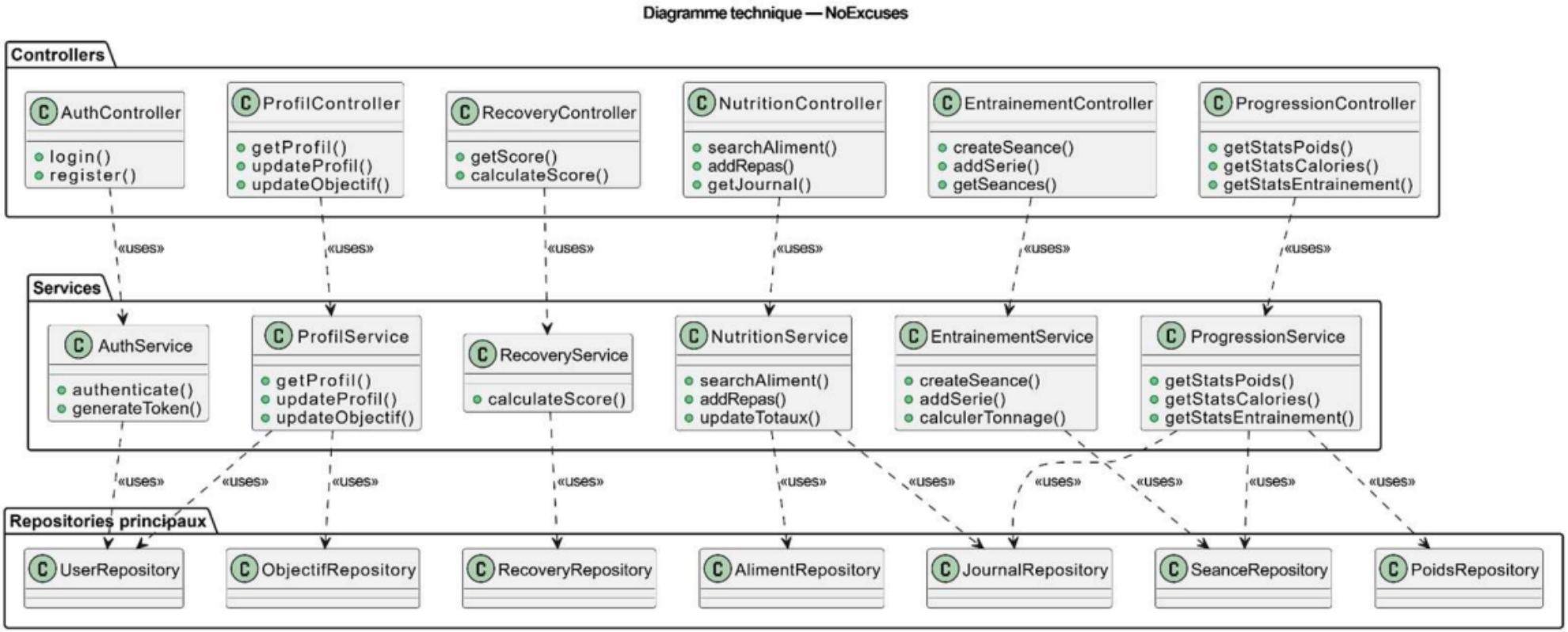


Fig. 12 - Organisation Controllers / Services / Repositories

PARTIE VIII

Architecture logicielle

34. Exigences techniques

L'application repose sur une architecture moderne répondant aux standards industriels actuels :

- **Architecture** : découplage complet entre le back-end (Symfony API) et le front-end (React SPA), assurant une séparation nette des responsabilités et une interface utilisateur très réactive.
- **Base de données** : persistance gérée par un système de gestion de base de données SQL (MySQL).
- **Conteneurisation** : Docker et Docker Compose pour l'orchestration des services (moteur PHP pour l'API Symfony, serveur web Nginx, base MySQL, environnement Node pour la compilation du front React), garantissant un environnement de développement et de production identique.
- **Qualité & CI/CD** : pipeline GitHub Actions automatisant les tests unitaires et d'intégration (PHPUnit) à chaque soumission de code.
- **Sécurité** : hachage robuste des mots de passe avec bcrypt, protection contre les failles critiques OWASP (injections SQL, XSS, CSRF) et conformité RGPD.

35. Pattern MVC avec Symfony

Le pattern MVC est appliqué à l'architecture découplée : React joue le rôle de vue côté client, tandis que Symfony gère les contrôleurs, la logique métier et l'accès aux données.

- **Contrôleurs** : point d'entrée de l'API, ils reçoivent les requêtes HTTP, valident les données et appellent le service métier correspondant.
- **Services** : ils contiennent la logique métier (calcul des calories, du tonnage, du score de récupération) et évitent de surcharger les contrôleurs.
- **Modèles & Repositories** : les entités Doctrine représentent les objets métier ; les repositories accèdent à la base de données.

36. Architecture n-tiers

NoExcuses suit une architecture logique en trois tiers, garantissant une séparation claire des responsabilités :

Tier	Rôle	Technologie
Présentation	Interface utilisateur	React (SPA)
Application	Reverse proxy + API et logique métier	Nginx + Symfony (PHP)
Données	Stockage persistant	MySQL

Le navigateur exécute l'application React, qui communique avec l'API Symfony via des requêtes HTTP au format JSON. Nginx joue le rôle de reverse proxy ; Symfony traite les requêtes, applique la logique métier et interagit avec la base via Doctrine. L'ensemble est conteneurisé avec Docker.

37. Composants externes et API

La stack intègre trois services externes ; les API métier CalorieNinjas et Mistral sont appelées côté serveur, tandis que la connexion Google implique le front-end (récupération du jeton) puis une validation côté back-end. Le module Nutrition s'appuie sur l'API **CalorieNinjas** couplée à l'IA **Mistral** pour la traduction FR/EN (et non sur OpenFoodFacts), retenue pour la qualité des données et la couverture des aliments en français.

Service externe	Rôle
CalorieNinjas	Recherche d'aliments et récupération des données nutritionnelles (mise en cache locale).
Mistral AI	Traduction FR/EN des recherches d'aliments et génération de variantes.
Google OAuth	Authentification déléguée : connexion via un compte Google.

PARTIE IX

Réalisation et développement

38. Organisation du code

Le projet suit une organisation en couches conforme au pattern MVC, séparée entre le back-end Symfony (API) et le front-end React (SPA).

Couche	Emplacement	Contenu
Contrôleurs (API)	backend/src/Controller	Auth, Nutrition, Entrainement, Recovery, Profil, Progression
Services (métier)	backend/src/Service	AuthService, NutritionService, EntrainementService, RecoveryService, ProfilService, ProgressionService, AiTranslationService
Entités & Repositories	backend/src/Entity, /Repository	11 entités Doctrine (User, Objectif, Aliment, JournalAlimentaire...) et leurs repositories
Pages React (SPA)	frontend/src/pages	Login, Register, Dashboard, Nutrition, Entrainement, Recovery, Progression, Profil
Composants & contexte	frontend/src/components, /context	Composants réutilisables (CalorieCircle, MacroBar...) et contexte d'authentification (JWT)

39. Principaux endpoints de l'API

L'API REST expose les points d'entrée suivants, tous protégés par JWT à l'exception de l'authentification :

Méthode(s)	Endpoint	Rôle
POST	/api/register, /api/login, /api/auth/google	Inscription, connexion, connexion Google
GET	/api/dashboard	Synthèse du jour (calories, macros, streak, recovery)
GET / POST / DELETE	/api/nutrition/aliments, /journal, /repas	Recherche d'aliments, journal, ajout/suppression de repas
GET / POST / PUT / DELETE	/api/seances, /seances/modeles, /seances/en-cours, /{id}/series	Modèles, séances, séries, tonnage, démarrer / terminer
GET / POST	/api/recovery/score, /sommeil	Score de récupération, historique de sommeil
GET	/api/progression/poids, /calories, /entrainement	Statistiques d'évolution
POST	/api/profil/poids	Ajout d'une pesée
GET	/api/profil/export/json, /export/excel	Export RGPD des données (JSON, Excel)

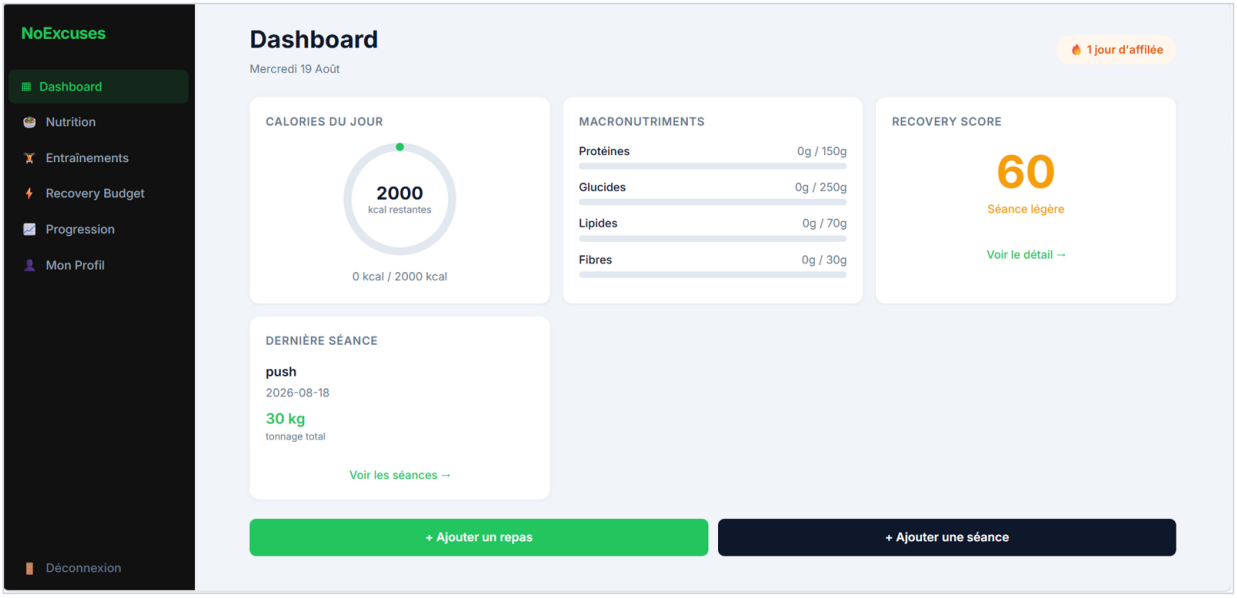
40. Modules développés

Les six modules du MVP sont pleinement implémentés et fonctionnels ; l'interface réalisée suit les maquettes haute fidélité présentées en Partie V (Fig. 5).

- **Authentification & Profil** : inscription et connexion (email/mot de passe ou compte Google), gestion du profil et des objectifs, et export RGPD des données personnelles (JSON et Excel).
- **Nutrition** : recherche d'aliments (CalorieNinjas + traduction Mistral), journal alimentaire du jour, calcul automatique des calories et macros.
- **Entraînement** : modèles de séances réutilisables (Push, Pull, Legs...), démarrage d'une séance, ajustement des exercices et séries, calcul du tonnage, et historique des séances terminées.
- **Recovery Budget** : saisie du sommeil, calcul du score de récupération (0-100) et recommandation associée.
- **Dashboard** : synthèse quotidienne (cercle de calories, barres de macros, streak, alerte de récupération).
- **Progression** : courbe d'évolution du poids et statistiques nutritionnelles et sportives.

Aperçu de l'interface réellement développée, fidèle aux maquettes haute fidélité de la Partie V :

Dashboard



Nutrition

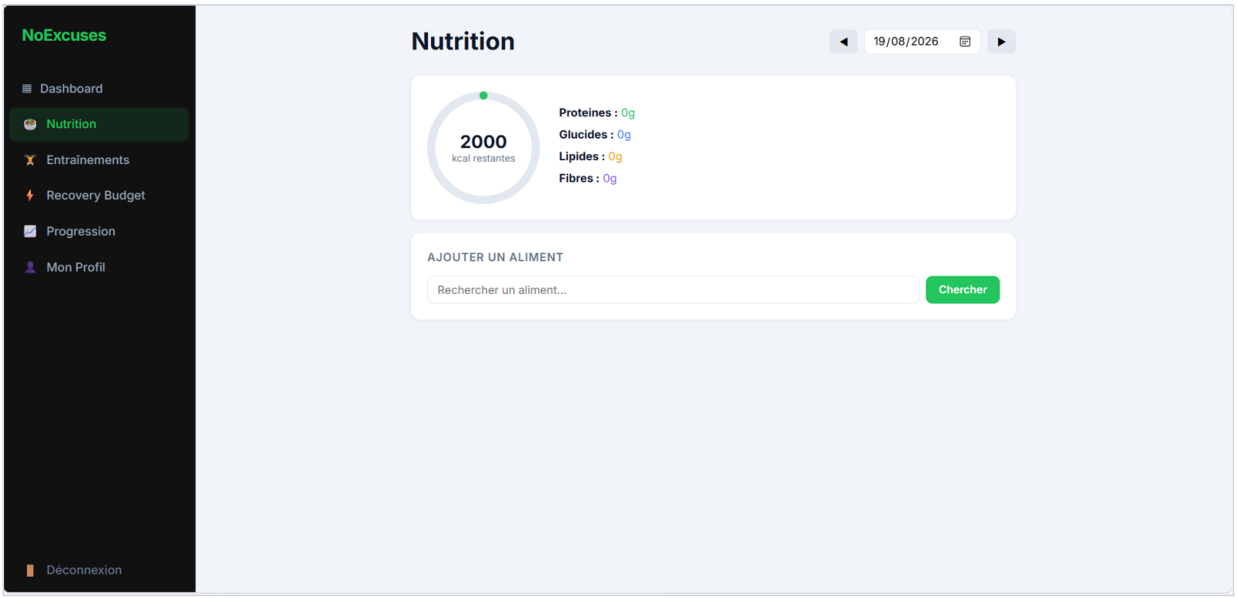
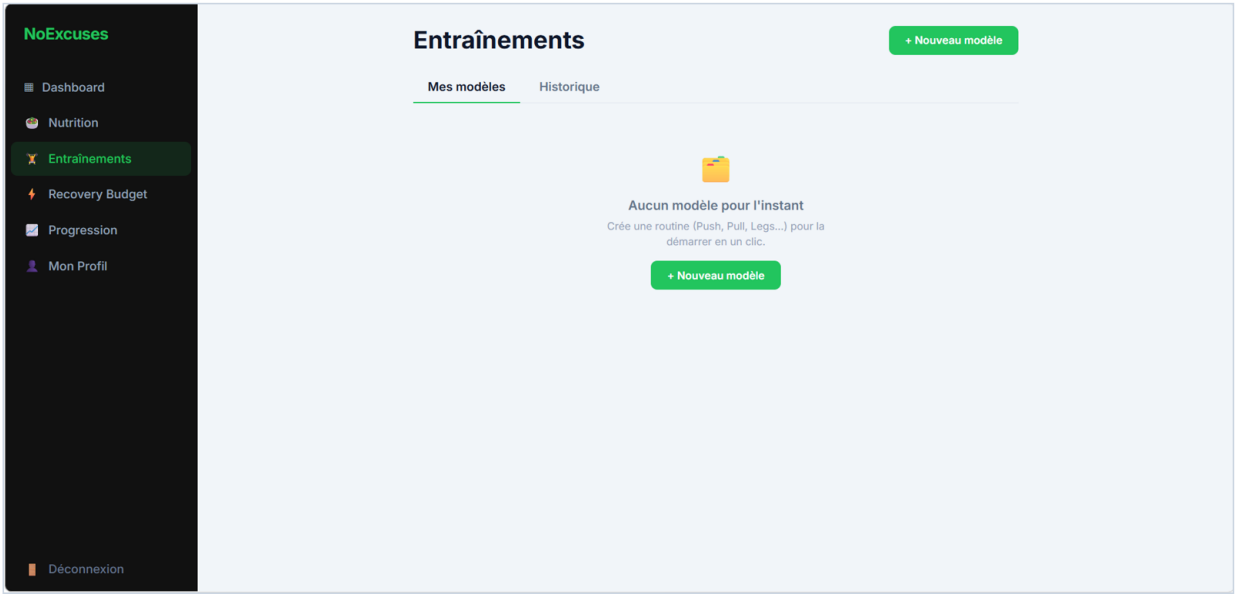


Fig. 13 - Interface de l'application : Dashboard et Nutrition

Entraînements



Progression

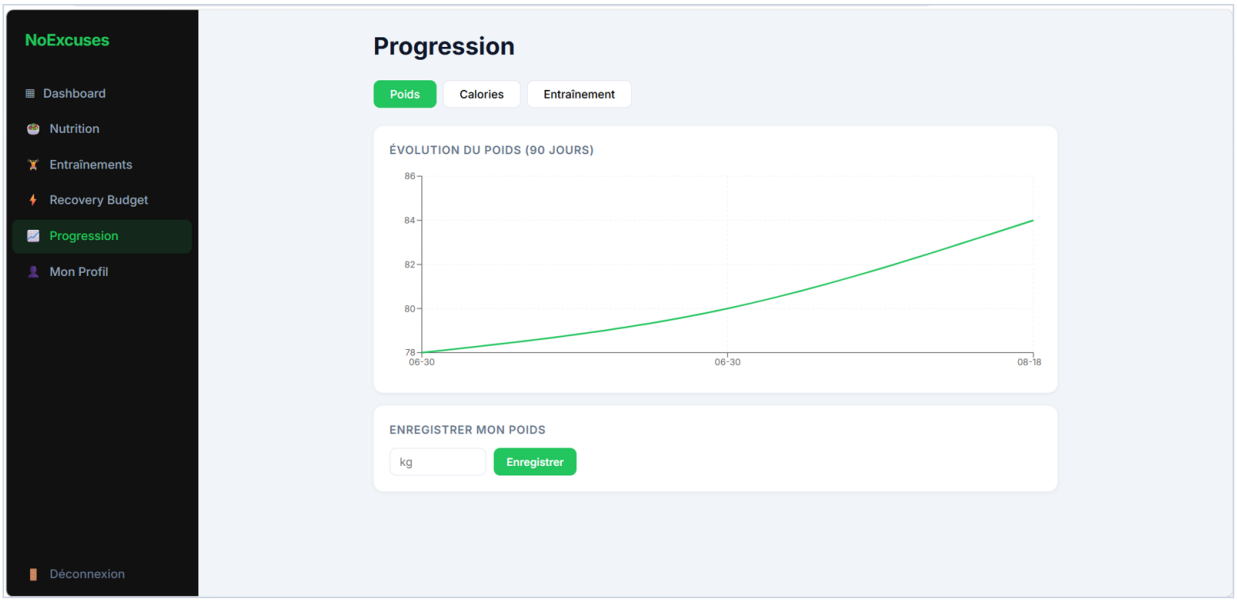


Fig. 14 - Interface de l'application : Entraînement et Progression

41. Extrait de code représentatif

Exemple de la logique d'authentification (AuthService) : la vérification du mot de passe s'appuie sur bcrypt, et un token JWT signé est émis en cas de succès. Ce token est ensuite transmis par le front-end dans le header Authorization pour accéder aux routes protégées.

```
public function login(string $email, string $password): string
{
    $user = $this->userRepository->findByEmail($email);

    // isPasswordValid compare le mot de passe fourni au hash bcrypt stocké
    if (!$user || !$this->hasher->isPasswordValid($user, $password)) {
        throw new \InvalidArgumentException('Identifiants invalides.');
    }

    // Génère un token JWT signé (RS256), renvoyé au client
    return $this->jwtManager->create($user);
}
```

PARTIE X

Sécurité

42. Analyse des menaces OWASP

42.1 Injection SQL

Menace : un attaquant injecte du code SQL via les paramètres d'entrée. **Protection** : l'intégralité des accès à la base passe par Doctrine ORM, qui utilise systématiquement des requêtes paramétrées (prepared statements) via son QueryBuilder. Les paramètres utilisateur ne sont jamais concaténés dans une requête SQL, ce qui prévient l'injection tant que les requêtes restent paramétrées.

42.2 Cross-Site Scripting (XSS)

Menace : injection de code JavaScript malveillant via les données affichées. **Protection** : React échappe nativement toutes les valeurs insérées dans le JSX ; l'utilisation de dangerouslySetInnerHTML est totalement absente du code. Aucune donnée utilisateur n'est injectée en HTML brut.

42.3 CSRF

Menace : un site tiers force le navigateur à effectuer des requêtes non désirées. **Protection** : l'API est sans état (stateless) et l'authentification repose sur un token JWT transmis dans le header Authorization, et non dans un cookie. Un site tiers ne pouvant pas injecter ce header en cross-origin, l'application n'est pas exposée au CSRF classique dans cette architecture.

43. Authentification : JWT, bcrypt et connexion déléguée

L'authentification classique repose sur un mot de passe haché avec **bcrypt** (coût 12, salt automatique, irréversible) et sur un **token JWT** signé (RS256, durée de vie limitée) transmis dans le header Authorization. Les routes /api/* sont protégées, à l'exception de /login, /register et /auth/google.

> **Authentification déléguée (Google OAuth 2.0)**
>
> L'application propose une connexion via Google. Le front-end récupère un token d'identité signé par Google et l'envoie au back-end, qui le fait valider par Google (vérification de l'audience et de l'email vérifié) avant de retrouver ou de créer le compte par email et d'émettre son propre JWT. Pour un compte créé via Google, un mot de passe aléatoire est généré et haché afin de respecter la contrainte NOT NULL du modèle, l'utilisateur se connectant exclusivement via Google.

Pour prévenir les attaques par force brute, l'endpoint **/login** est protégé par une limitation du nombre de tentatives (Symfony Rate Limiter) : au-delà de cinq essais infructueux sur une fenêtre glissante de quinze minutes pour un même couple IP + email, les requêtes sont rejetées (HTTP 429 Too Many Requests). Le compteur est réinitialisé après une connexion réussie.

44. CORS, validation et protection des données (RGPD)

Le bundle NelmioCorsBundle restreint les origines autorisées à appeler l'API (localhost en développement, domaine de production uniquement en production). Le composant Symfony Validator valide les données entrantes (email, longueur du mot de passe, plages de valeurs physiologiques). Les clés API et secrets sont stockés dans des fichiers .env non versionnés. Le serveur Nginx ajoute par

ailleurs des en-têtes HTTP de sécurité (X-Frame-Options contre le clickjacking, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy).

Côté RGPD, l'application ne collecte que les données nécessaires au service (email, prénom, nom, données de santé). Chaque donnée est liée à l'utilisateur connecté et inaccessible aux autres, et la suppression d'un compte entraîne la suppression en cascade de toutes ses données. Les données transmises aux prestataires se limitent au strict nécessaire : le nom de l'aliment recherché (CalorieNinjas), le terme à traduire (Mistral) et le jeton d'authentification (Google).

Conformément au droit à la portabilité (RGPD), l'utilisateur peut **télécharger une copie complète de ses données** depuis son profil, en JSON (structuré) ou en Excel multi-feuilles (lisible). L'export est généré côté serveur à partir de l'utilisateur authentifié (jamais d'identifiant fourni par le client) et ne contient aucune donnée sensible (mot de passe, jetons, secrets) ; seules les séances terminées y figurent.

45. Principaux risques OWASP retenus

Menace OWASP	Mesure appliquée
A01 - Broken Access Control	JWT sur toutes les routes privées, vérification du propriétaire des données.
A02 - Cryptographic Failures	bcrypt (coût 12), JWT signé RSA 2048, HTTPS en production.
A03 - Injection (dont XSS)	Doctrine ORM et requêtes paramétrées ; React échappe nativement les sorties (XSS).
A05 - Security Misconfiguration	CORS restreint, secrets hors dépôt Git, en-têtes HTTP de sécurité (Nginx).
A07 - Auth Failures	JWT signé RS256, durée de vie limitée ; limitation des tentatives de connexion (5 / 15 min) contre le brute force.

PARTIE XI

Tests et qualité

46. Stratégie de tests

La stratégie combine deux niveaux complémentaires. Les tests unitaires valident la logique métier en isolation avec des mocks (chaque dépendance, EntityManager ou Repository, est simulée), ce qui les rend rapides et reproductibles. Les tests d'intégration valident le comportement réel de l'application face à de vraies requêtes HTTP, sur une base MySQL de test.

Type	Outil	Périmètre	Résultat
Tests unitaires	PHPUnit	Services métier (Auth, Entraînement, Nutrition, Recovery)	13 tests
Tests d'intégration	PHPUnit WebTestCase	Endpoints de l'API sur base MySQL de test	15 tests
Build front-end	React / npm	Compilation de production	OK

47. Tests unitaires (PHPUnit)

Les tests unitaires sont répartis par module métier :

- **AuthServiceTest** : création d'un compte avec un email libre, et levée d'exception si l'email est déjà utilisé.
- **EntrainementServiceTest** : calcul du tonnage (5 répétitions × 100 kg = 500 kg), agrégation du tonnage d'une séance, archivage d'une séance, statut « en cours » par défaut à la création, et reconnaissance d'une séance-modèle.
- **NutritionServiceTest** : calcul des calories d'une ligne (150 g de poulet grillé = 247,5 kcal) et arrondi correct à deux décimales.
- **RecoveryServiceTest** : score élevé en conditions optimales, score de repos en cas de surentraînement, et recommandations (séance normale, repos).

48. Tests d'intégration

Les tests d'intégration envoient de vraies requêtes HTTP aux endpoints de l'AuthController, sur une base MySQL de test recréée à chaque exécution. Ils valident le parcours complet Contrôleur → Service → base de données :

Scénario testé	Vérification	Réponse
Inscription	Création d'un compte avec un email disponible	201
Email en doublon	Refus d'un email déjà utilisé	409
Connexion valide	Identifiants corrects, un token JWT est retourné	200
Mauvais mot de passe	Connexion refusée	401
Route protégée	Accès à /api/profil sans token JWT bloqué	401

Scénario testé	Vérification	Réponse
Connexion Google sans jeton	Refus d'une connexion Google sans identifiant	400
Anti brute force	Blocage après 5 tentatives de connexion échouées	429
Export sans jeton	Export des données personnelles bloqué sans authentification	401
Export authentifié	Téléchargement JSON et Excel des données	200
Confidentialité de l'export	Ni donnée sensible, ni donnée d'un autre utilisateur	OK

Ces tests prouvent concrètement que la sécurité fonctionne (une route protégée renvoie bien 401 sans authentification) et que tout le flux d'authentification est opérationnel de bout en bout.

49. Intégration continue

La chaîne GitHub Actions exécute l'ensemble des tests (unitaires et d'intégration) et le build React à chaque push sur develop et à chaque Pull Request vers main. Le pipeline garantit qu'une version cassée ne peut pas être déployée, assurant la stabilité de l'application à chaque étape.

PARTIE XII

Déploiement et mise en production

50. Conteneurisation Docker

L'application est entièrement conteneurisée avec Docker, ce qui permet de la lancer sur n'importe quelle machine avec une seule commande, sans installation manuelle de PHP, MySQL ou Node. La conteneurisation répond à un objectif clé : la **reproductibilité**, le même environnement tournant à l'identique en développement, en test et en production, ce qui élimine le classique « ça marche sur mon poste ».

Conteneur	Image	Rôle	Port
mysql	mysql:8.0 (officielle)	Base de données	3306
symfony	construite (Dockerfile)	API PHP 8.3 (PHP-FPM)	9000
nginx	nginx:1.27 (officielle)	Reverse proxy du backend	8000
react	construite (Dockerfile)	Frontend SPA	3000
phpmyadmin	phpmyadmin (officielle)	Administration base (dev)	8081

Les conteneurs communiquent sur un réseau Docker privé en s'appelant par leur nom (Symfony joint la base via mysql:3306). Les données MySQL sont conservées dans un volume nommé, préservé même si le conteneur est recréé.

51. Environnements de développement et production

L'application est identique entre développement et production ; seule la configuration change (la production est plus fermée, plus sécurisée et optimisée). Deux fichiers d'orchestration distincts gèrent ces différences.

Aspect	Développement	Production
Base MySQL	Port exposé	Non exposée (réseau interne)
phpMyAdmin	Présent	Retiré
APP_ENV	dev (debug actif)	prod (debug off, cache optimisé)
Code	Monté en volume (live)	Figé dans l'image
Frontend	Serveur de dev (npm)	Compilé, servi par Nginx
Dépendances PHP	Toutes (avec dev)	composer --no-dev optimisé

52. Déploiement et stratégie de mise en production

Le déploiement s'effectue par une procédure automatisée : les conteneurs se construisent, la base est initialisée par les migrations et l'application démarre seule. Un script d'entrypoint attend que la base soit prête, applique les migrations, génère les clés JWT si nécessaire, puis lance le serveur, sans étape

manuelle. Plusieurs stratégies de mise à jour sans coupure existent (déploiement bleu/vert, rolling update) ; pour ce projet, une fenêtre de maintenance suffit, la chaîne CI/CD constituant la première brique d'un déploiement continu.

53. Retour d'expérience DevOps

La mise en place de Docker a nécessité de résoudre plusieurs problèmes concrets, caractéristiques du passage à la conteneurisation :

Problème rencontré	Solution appliquée
Symfony tentait de joindre la base sur 127.0.0.1 (le conteneur lui-même)	Utiliser le nom de service « mysql » sur le réseau Docker
Collision des noms d'images entre dev et prod	Nom de projet distinct (name: noexcuses-prod) dans le compose prod
Composer désactive ses plugins en exécution root	Variable COMPOSER_ALLOW_SUPERUSER=1
Chemin relatif des clés JWT invalide en requête web	Chemin absolu via %kernel.project_dir%
Certificat SSL auto-signé de MySQL 8 refusé	Option --skip-ssl pour la vérification de disponibilité

Avoir rencontré et résolu ces difficultés a permis de mieux comprendre le fonctionnement réseau de Docker, la gestion des secrets et les différences entre environnements de développement et de production.

Conclusion

NoExcuses est une application web complète qui répond au besoin initial : centraliser et simplifier le suivi sportif et nutritionnel. Le projet a été mené de bout en bout, de l'analyse du besoin à la mise en production, en respectant une démarche structurée : cahier des charges fonctionnel, modélisation MERISE, conception UML, développement en architecture découplée (Symfony / React), sécurisation (OWASP, JWT, bcrypt, OAuth), tests automatisés et conteneurisation Docker avec intégration continue.

Les six modules fonctionnels du MVP sont opérationnels, la base de données est normalisée (3NF) et cohérente, et la qualité est garantie par une chaîne de tests et d'intégration continue. Le projet démontre la maîtrise de l'ensemble du cycle de vie d'une application, de la conception au déploiement. Mené intégralement en autonomie, il consolide dans une réalisation concrète et fonctionnelle l'ensemble des compétences du métier de Concepteur Développeur d'Applications.

Des évolutions sont envisagées pour prolonger le projet : augmentation de la couverture de tests, tests de composants React, et notifications et rappels quotidiens.

Annexes

Les annexes ci-dessous reprennent, tels qu'établis lors de la modélisation MERISE, le dictionnaire des données complet (Annexe A) et le script SQL de création de la base sous MySQL (Annexe B).

Annexe A — Dictionnaire des données :

Entité	Attribut	Type générique	Description	Contrainte
utilisateur	id	Entier	Identifiant unique auto-incrémenté	PK, NOT NULL
utilisateur	email	Texte (255)	Adresse email de connexion	UNIQUE, NOT NULL
utilisateur	mot_de_passe	Texte (255)	Mot de passe haché (bcrypt)	NOT NULL
utilisateur	prenom	Texte (100)	Prénom de l'utilisateur	NOT NULL
utilisateur	nom	Texte (100)	Nom de famille	NOT NULL
utilisateur	date_naissance	Date	Date de naissance	NULL
utilisateur	sexe	Texte (10)	Sexe biologique (homme/femme)	NULL
utilisateur	taille	Entier	Taille en centimètres	NULL
utilisateur	poids_initial	Décimal (5,2)	Poids de départ en kg	NULL
utilisateur	calories_objectif	Entier	Objectif calorique journalier	NULL
utilisateur	niveau_activite	Texte (50)	Niveau d'activité (sédentaire, actif...)	NULL
utilisateur	created_at	DateTime	Date de création du compte	NOT NULL
utilisateur	updated_at	DateTime	Date de dernière mise à jour	NULL
objectif	id	Entier	Identifiant unique	PK, NOT NULL
objectif	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL
objectif	type_objectif	Texte (50)	Type : perte de poids, prise de masse, maintien	NOT NULL
objectif	calories_cible	Entier	Calories journalières cibles	NOT NULL
objectif	proteines_cible	Décimal (5,2)	Objectif protéines en grammes	NULL
objectif	glucides_cible	Décimal (5,2)	Objectif glucides en grammes	NULL
objectif	lipides_cible	Décimal (5,2)	Objectif lipides en grammes	NULL
objectif	poids_cible	Décimal (5,2)	Poids visé en kg	NULL
objectif	date_debut	Date	Date de début de l'objectif	NOT NULL
objectif	date_fin	Date	Date de fin prévue	NULL
aliment	id	Entier	Identifiant unique local	PK, NOT NULL
aliment	nom	Texte (255)	Nom de l'aliment	NOT NULL

aliment	code_api	Texte (100)	Identifiant dans l'API nutritionnelle externe	NULL, UNIQUE
aliment	calories_100g	Décimal (6,2)	Calories pour 100g	NOT NULL
aliment	proteines_100g	Décimal (6,2)	Protéines pour 100g en grammes	NULL
aliment	glucides_100g	Décimal (6,2)	Glucides pour 100g en grammes	NULL
aliment	lipides_100g	Décimal (6,2)	Lipides pour 100g en grammes	NULL
aliment	fibres_100g	Décimal (6,2)	Fibres pour 100g en grammes	NULL
journal_alimentaire	id	Entier	Identifiant unique	PK, NOT NULL
journal_alimentaire	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL
journal_alimentaire	date_journal	Date	Date du journal alimentaire	NOT NULL
journal_alimentaire	calories_totales	Décimal (7,2)	Total calories consommées (calculé)	NULL
journal_alimentaire	proteines_totales	Décimal (6,2)	Total protéines consommées (calculé)	NULL
journal_alimentaire	glucides_totaux	Décimal (6,2)	Total glucides consommés (calculé)	NULL
journal_alimentaire	lipides_totaux	Décimal (6,2)	Total lipides consommés (calculé)	NULL
journal_alimentaire	fibres_totales	Décimal (6,2)	Total fibres consommées (calculé)	NULL
ligne_journal	id	Entier	Identifiant unique	PK, NOT NULL
ligne_journal	id_journal	Entier	Référence au journal alimentaire	FK -> JournalAlimentaire, NOT NULL
ligne_journal	id_aliment	Entier	Référence à l'aliment	FK -> Aliment, NOT NULL
ligne_journal	type_repas	Texte (50)	Petit-déjeuner, Déjeuner, Dîner, En-cas	NOT NULL
ligne_journal	quantite	Décimal (6,2)	Quantité consommée en grammes	NOT NULL
ligne_journal	calories	Décimal (6,2)	Calories pour la quantité saisie (calculé)	NOT NULL
seance	id	Entier	Identifiant unique	PK, NOT NULL
seance	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL
seance	nom	Texte (150)	Nom de la séance (ex: Push, Pull, Legs)	NOT NULL
seance	date_seance	DateTime	Date et heure de la séance	NOT NULL
seance	duree_minutes	Entier	Durée de la séance en minutes	NULL
seance	tonnage_total	Décimal (8,2)	Somme des charges soulevées (calculé)	NULL
seance	notes	Texte long	Notes libres sur la séance	NULL

seance	statut	Texte (20)	En cours, Terminée, Archivée	NOT NULL
exercice	id	Entier	Identifiant unique	PK, NOT NULL
exercice	nom	Texte (150)	Nom de l'exercice (ex: Développé couché)	NOT NULL
exercice	groupe_musculaire	Texte (100)	Groupe musculaire ciblé	NULL
exercice	description	Texte long	Description et conseils d'exécution	NULL
serie_exercice	id	Entier	Identifiant unique	PK, NOT NULL
serie_exercice	id_seance	Entier	Référence à la séance	FK -> Seance, NOT NULL
serie_exercice	id_exercice	Entier	Référence à l'exercice	FK -> Exercice, NOT NULL
serie_exercice	numero_serie	Entier	Numéro de la série dans la séance	NOT NULL
serie_exercice	repetitions	Entier	Nombre de répétitions effectuées	NOT NULL
serie_exercice	charge_kg	Décimal (5,2)	Charge utilisée en kilogrammes	NULL
serie_exercice	tonnage	Décimal (7,2)	Tonnage = reps x charge (calculé)	NULL
sommeil	id	Entier	Identifiant unique	PK, NOT NULL
sommeil	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL
sommeil	date_nuit	Date	Date de la nuit de sommeil	NOT NULL
sommeil	heures_sommeil	Décimal (3,1)	Nombre d'heures de sommeil déclarées	NOT NULL
sommeil	qualite_sommeil	Entier	Score de qualité de 1 à 5 (déclaratif)	NULL
recovery_score	id	Entier	Identifiant unique	PK, NOT NULL
recovery_score	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL
recovery_score	date_calcul	Date	Date du calcul du score	NOT NULL
recovery_score	score	Entier	Score de récupération de 0 à 100	NOT NULL
recovery_score	charge_7j	Décimal (9,2)	Tonnage cumulé sur les 7 derniers jours	NULL
recovery_score	bilan_calorique	Décimal (7,2)	Bilan calorique du jour (consommé - objectif)	NULL
recovery_score	heures_sommeil_moy	Décimal (3,1)	Moyenne de sommeil sur 7 jours	NULL
recovery_score	recommandation	Texte (50)	Repos recommandé / Séance légère / Séance normale	NULL
poids_historique	id	Entier	Identifiant unique	PK, NOT NULL
poids_historique	id_utilisateur	Entier	Référence à l'utilisateur	FK -> Utilisateur, NOT NULL

poids_historique	date_mesure	Date	Date de la pesée	NOT NULL
poids_historique	poids_kg	Décimal (5,2)	Poids mesuré en kilogrammes	NOT NULL
poids_historique	notes	Texte (255)	Commentaire éventuel (ex: après repas)	NULL

Annexe B — Script SQL MPD

```
MPD: SET FOREIGN_KEY_CHECKS = 0;

SET NAMES utf8mb4;

-- ============================================================
-- TABLE : utilisateur
-- ============================================================
CREATE TABLE IF NOT EXISTS utilisateur (
    id              INT AUTO_INCREMENT NOT NULL,
    email           VARCHAR(255)       NOT NULL,
    mot_de_passe    VARCHAR(255)       NOT NULL,
    prenom          VARCHAR(100)       NOT NULL,
    nom             VARCHAR(100)       NOT NULL,
    date_naissance  DATE               NULL,
    sexe            VARCHAR(10)        NULL,
    taille          INT                NULL,
    poids_initial   DECIMAL(5,2)       NULL,
    calories_objectif INT              NULL,
    niveau_activite VARCHAR(50)        NULL,
    created_at      DATETIME           NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    updated_at      DATETIME           NULL     ON UPDATE
CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_utilisateur_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ============================================================
-- TABLE : objectif
-- ============================================================
CREATE TABLE IF NOT EXISTS objectif (
    id              INT AUTO_INCREMENT NOT NULL,
    id_utilisateur  INT                NOT NULL,
    type_objectif   VARCHAR(50)        NOT NULL,
    calories_cible  INT                NOT NULL,
    proteines_cible DECIMAL(5,2)       NULL,
    glucides_cible  DECIMAL(5,2)       NULL,
    lipides_cible   DECIMAL(5,2)       NULL,
    poids_cible     DECIMAL(5,2)       NULL,
    date_debut      DATE               NOT NULL,
    date_fin        DATE               NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_objectif_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================================
-- TABLE : aliment
-- ============================================================
CREATE TABLE IF NOT EXISTS aliment (
    id              INT AUTO_INCREMENT NOT NULL,
    nom             VARCHAR(255)       NOT NULL,
    code_api        VARCHAR(100)       NULL,
    calories_100g   DECIMAL(6,2)       NOT NULL,
    proteines_100g  DECIMAL(6,2)       NULL,
    glucides_100g   DECIMAL(6,2)       NULL,
    lipides_100g    DECIMAL(6,2)       NULL,
    fibres_100g     DECIMAL(6,2)       NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_aliment_code_api (code_api)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ============================================================
-- TABLE : journal_alimentaire
-- ============================================================
CREATE TABLE IF NOT EXISTS journal_alimentaire (
    id                  INT AUTO_INCREMENT NOT NULL,
    id_utilisateur      INT                NOT NULL,
    date_journal        DATE               NOT NULL,
    calories_totales    DECIMAL(7,2)       NULL,
    proteines_totales   DECIMAL(6,2)       NULL,
    glucides_totaux     DECIMAL(6,2)       NULL,
    lipides_totaux      DECIMAL(6,2)       NULL,
    fibres_totales      DECIMAL(6,2)       NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_journal_user_date (id_utilisateur, date_journal),
    CONSTRAINT fk_journal_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================================
-- TABLE : ligne_journal
-- ============================================================
CREATE TABLE IF NOT EXISTS ligne_journal (
    id          INT AUTO_INCREMENT NOT NULL,
    id_journal  INT                NOT NULL,
    id_aliment  INT                NOT NULL,
    type_repas  VARCHAR(50)        NOT NULL,
    quantite    DECIMAL(6,2)       NOT NULL,
    calories    DECIMAL(6,2)       NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_ligne_journal
        FOREIGN KEY (id_journal) REFERENCES journal_alimentaire (id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ligne_aliment
        FOREIGN KEY (id_aliment) REFERENCES aliment (id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ============================================================
-- TABLE : seance
-- ============================================================
CREATE TABLE IF NOT EXISTS seance (
    id              INT AUTO_INCREMENT NOT NULL,
    id_utilisateur  INT                NOT NULL,
    nom             VARCHAR(150)       NOT NULL,
    date_seance     DATETIME           NOT NULL,
    duree_minutes   INT                NULL,
    tonnage_total   DECIMAL(8,2)       NULL,
    statut          VARCHAR(20)        NOT NULL DEFAULT 'en_cours',
    notes           TEXT               NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_seance_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
-- ============================================================
-- TABLE : exercice
-- ============================================================
CREATE TABLE IF NOT EXISTS exercice (
    id                  INT AUTO_INCREMENT NOT NULL,
    nom                 VARCHAR(150)       NOT NULL,
    groupe_musculaire   VARCHAR(100)       NULL,
    description         TEXT               NULL,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================================
-- TABLE : serie_exercice
-- ============================================================
CREATE TABLE IF NOT EXISTS serie_exercice (
    id              INT AUTO_INCREMENT NOT NULL,
    id_seance       INT                NOT NULL,
    id_exercice     INT                NOT NULL,
    numero_serie    INT                NOT NULL,
    repetitions     INT                NOT NULL,
    charge_kg       DECIMAL(5,2)       NULL,
    tonnage         DECIMAL(7,2)       NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_serie_seance
        FOREIGN KEY (id_seance) REFERENCES seance (id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_serie_exercice
        FOREIGN KEY (id_exercice) REFERENCES exercice (id)
```

```
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================================
-- TABLE : sommeil
-- ============================================================
CREATE TABLE IF NOT EXISTS sommeil (
    id              INT AUTO_INCREMENT NOT NULL,
    id_utilisateur  INT                NOT NULL,
    date_nuit       DATE               NOT NULL,
    heures_sommeil  DECIMAL(3,1)       NOT NULL,
    qualite_sommeil INT                NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_sommeil_user_date (id_utilisateur, date_nuit),
    CONSTRAINT fk_sommeil_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================================
-- TABLE : recovery_score
-- ============================================================
CREATE TABLE IF NOT EXISTS recovery_score (
    id                  INT AUTO_INCREMENT NOT NULL,
    id_utilisateur      INT                NOT NULL,
    date_calcul         DATE               NOT NULL,
    score               INT                NOT NULL,
    charge_7j           DECIMAL(9,2)       NULL,
    bilan_calorique     DECIMAL(7,2)       NULL,
    heures_sommeil_moy  DECIMAL(3,1)       NULL,
    recommandation      VARCHAR(50)        NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_recovery_user_date (id_utilisateur, date_calcul),
    CONSTRAINT fk_recovery_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ============================================================
-- TABLE : poids_historique
-- ============================================================
CREATE TABLE IF NOT EXISTS poids_historique (
    id              INT AUTO_INCREMENT NOT NULL,
    id_utilisateur  INT                NOT NULL,
    date_mesure     DATE               NOT NULL,
    poids_kg        DECIMAL(5,2)       NOT NULL,
    notes           VARCHAR(255)       NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_poids_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur (id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;
```